
Aufgabendatenbank

Projektdokumentation

Ein Werkzeug zum Beschreiben, Strukturieren und Speichern
von Lernaufgaben

Betreuung: Prof. Dr. Markus Siepermann
Johannes Kunz

Projektteam: [Vorname Nachname] (*Projektleitung*)
[Vorname Nachname]
[Vorname Nachname]

Gruppe: [Gruppennummer]

Abgabe: 14. Juli 2026

Inhaltsverzeichnis

1	Kurze Zusammenfassung	3
2	Aufgabenstellung und Zielsetzung	3
2.1	Aufgabenstellung	3
2.2	Abgrenzung des Funktionsumfangs	3
2.3	Zielsetzung	3
3	Ausgangssituation und Recherche	4
3.1	Ausgangssituation	4
3.2	Technologieauswahl und Begründung	4
4	Anforderungen und Rahmenbedingungen	5
4.1	Funktionale Anforderungen	5
4.2	Nicht-funktionale Anforderungen	5
4.3	Rahmenbedingungen	5
5	Konzept und Entwurf	6
5.1	Systemarchitektur	6
5.2	Datenmodell: die Entitäten	6
5.2.1	Referenz-Entitäten (Nachschlagelisten)	6
5.2.2	Kern-Entitäten	7
5.2.3	Verknüpfungs-Entitäten (n:m-Beziehungen)	7
5.3	Beziehungskonzept: vertikal und horizontal	7
5.4	Referentielle Integrität	8
6	Umsetzung	8
6.1	Aufgabe erstellen (durchgängiger Ablauf)	8
6.2	Inhalte in mehreren Formaten	9
6.3	Kollektionen und Reihenfolge	9
6.4	Horizontale Varianten	9
6.5	Validatoren	9
6.6	Referenzdaten auswählen und anlegen	9
6.7	Hierarchische Tags, Filter und Suche	9
6.8	Darstellungs-Templates	10
6.9	Qualitätssicherung	10
6.10	Grundlage für den Cluster-Betrieb	10
7	Projektorganisation	10
7.1	Vorgehen und Versionsverwaltung	10
7.2	Zusammenarbeit im Team	10
7.3	Projektplanung: Phasen und Meilensteine	11
7.4	Aufgabenverteilung	11
7.5	Abweichungen vom ursprünglichen Plan	12
8	Grenzen und Ausblick	12
8.1	Nicht realisiert (mit Begründung)	12
8.2	Bewusst zurückgestellt	12
8.3	Bekannte technische Grenzen	12
8.4	Ausblick	12
9	Fazit	13

1 Kurze Zusammenfassung

Ausgangspunkt des Projekts war ein vorgegebenes relationales Datenbankschema für eine *Aufgabendatenbank*. Ziel war es, auf dieser Grundlage eine vollständige, lauffähige Webanwendung zu entwickeln, mit der Lehrende Aufgaben strukturiert erfassen, mit Metadaten versehen und untereinander in Beziehung setzen können.

Umgesetzt wurde eine Drei-Schichten-Anwendung aus einem Vue-Frontend, einem Spring-Boot-Backend und einer PostgreSQL-Datenbank. Alle zentralen fachlichen Konzepte des Schemas sind implementiert: Aufgaben mit Metadaten (Autor, Lizenz, Typ), Inhaltsbausteine in mehreren Formaten (Text, Bild, PDF), geordnete und ungeordnete Kollektionen, horizontale Varianten, Validatoren, hierarchische Tags mit Filter- und Suchfunktion sowie Darstellungs-Templates. Für den Betrieb auf der E-Learning-Plattform der Hochschule wurde eine automatisierte Build-Pipeline (CI → GHCR) samt Konfigurationsdateien vorbereitet.

Das Projektteam bestand zu Beginn aus fünf Personen und wurde nach dem Ausstieg zweier Mitglieder in der Anfangsphase auf drei Personen verkleinert; der Funktionsumfang wurde vollständig auf dieses Team umverteilt und umgesetzt. Nicht realisiert wurden das eigentliche Cluster-Deployment (das zuständige Infrastruktur-Team stellte seine Arbeit ein) sowie eine reale Authentifizierung und eine ursprünglich angedachte KI-Funktion (siehe Kapitel 8).

2 Aufgabenstellung und Zielsetzung

2.1 Aufgabenstellung

Die Aufgabe bestand darin, aus einem vorgegebenen relationalen Datenmodell eine funktionsfähige Anwendung zu entwickeln. Das Schema definierte die zentralen Entitäten und ihre Beziehungen; die konkrete Anwendung — Frontend, Backend und Persistenzschicht — war vom Team zu entwerfen und zu implementieren. Es handelte sich also nicht um ein Projekt „auf der grünen Wiese“, sondern um die fachlich korrekte Umsetzung einer vorgegebenen Datenstruktur in eine benutzbare Software.

2.2 Abgrenzung des Funktionsumfangs

Eine frühe, für das gesamte Projekt prägende Klärung mit dem Betreuer betraf die Abgrenzung: Die Anwendung ist ausdrücklich ein **Autoren- und Verwaltungswerkzeug** und *kein* Lern- oder Prüfungssystem. Sie unterstützt das Erstellen und Organisieren von Aufgaben, nicht deren Bearbeitung durch Lernende.

- **Im Verantwortungsbereich:** Aufgaben erfassen, mit Metadaten versehen, in Sequenzen und Varianten in Beziehung setzen sowie Prüfregele (Validatoren) *definieren und speichern*.
- **Nicht im Verantwortungsbereich:** das eigentliche Lösen oder Korrigieren von Aufgaben sowie die automatische Erzeugung von Varianten. Diese Ausführung übernimmt ein separater Dienst der Plattform.
- **Zugriffsmodell:** In dieser Iteration gibt es bewusst keine Rechteeinschränkung; jeder Nutzer kann alles anlegen und bearbeiten. Eine spätere zentrale Anmeldung (THM-CAS) kann darauf aufsetzen.

2.3 Zielsetzung

Angestrebt wurde eine fachlich korrekte, technisch saubere und erweiterbare Umsetzung des Datenmodells mit einer benutzerfreundlichen Oberfläche: alle Kern-Metadatenkonzepte abbilden,

die beiden im Schema angelegten Beziehungsarten (vertikal und horizontal) unterstützen und die Anwendung für den Betrieb auf der Hochschulplattform vorbereiten.

3 Ausgangssituation und Recherche

3.1 Ausgangssituation

Zu Projektbeginn lagen das relationale Schema und eine grobe fachliche Beschreibung vor. Die erste Aufgabe war daher weniger das Programmieren als das *Verstehen* des Modells. Zwei Punkte erwiesen sich als zentral und mussten im Team gemeinsam erarbeitet werden:

- **Eine Kollektion ist selbst ein Item.** Es gibt keine getrennte „Sammlung“ neben der „Aufgabe“; über `parent_item_id` ist eine Kollektion zugleich ein Item und teilt sich dessen Metadaten und Verknüpfungen.
- **Es gibt zwei getrennte Beziehungsarten.** Das Schema kennt eine *vertikale* (geordnete Sequenzen) und eine *horizontale* (Varianten über ein gemeinsames Wurzel-Item) Beziehung. Beide dürfen nicht vermischt werden — eine Erkenntnis, die später eine wichtige Entwurfsentscheidung nach sich zog (Abschnitt 5.3).

3.2 Technologieauswahl und Begründung

Auf Basis dieses Verständnisses wurden geeignete Technologien recherchiert. Die Auswahl orientierte sich an drei Kriterien: Eignung für die fachliche Aufgabe, durchgängige Typsicherheit sowie gute Container- und Cluster-Tauglichkeit, da die Plattform der Hochschule auf Kubernetes aufbaut. Neben der reinen Eignung waren zwei praktische Gründe ausschlaggebend: der Betreuer stellte eine **Vue-Vorlage** als Startpunkt bereit, und das Team hatte bereits **Vorerfahrung** mit dem gewählten Stack (Vue, Spring Boot, PostgreSQL).

Technologie	Version	Eingesetzt für
Vue (Composition API)	3.5.34	reaktives Komponenten-Frontend
Vuetify	4.0.7	Material-Design-UI (Formulare, Dialoge)
Pinia	3.0.4	zentraler, typisierter Zustand (Store)
vuedraggable	4.1.0	Drag-and-Drop im Strukturbaum
CodeMirror 6 (vue-codemirror)	6.x	XML-Editor für Darstellungs-Templates
axios	1.6	HTTP-Kommunikation mit dem Backend
Vite / Vitest	8.0.12 / 4.1.6	Build-Werkzeug / Test-Framework
TypeScript	6.0	Typsicherheit über alle Frontend-Schichten
Spring Boot	3.2.5	REST-Backend (Controller/Service/Repository)
Java	21	Backend-Sprache
Spring Data JPA	(Boot 3.2.5)	deklarative Persistenz auf dem Schema
PostgreSQL	16	relationale Datenbank
Docker Compose	—	reproduzierbarer lokaler Start

Frontend — Vue, Vuetify, Pinia. Vue 3 mit der Composition API bietet ein reaktives Komponentenmodell, das sich gut für die zentrale Baum-/Editor- Ansicht eignet. Ausschlaggebend waren zusätzlich die vom Betreuer bereitgestellte Vue-Vorlage und die vorhandene Team-Erfahrung. Vuetify liefert eine konsistente Komponentenbibliothek und reduziert den Aufwand für Formulare und Dialoge; Pinia hält den gesamten Anwendungszustand in einem typisierten Store. Für zwei spezielle Anforderungen kamen dedizierte Bibliotheken zum Einsatz: **vuedraggable** für die Drag-and-Drop-Umsortierung im Strukturbaum und **CodeMirror 6** (über `vue-codemirror`) als Editor für die XML-Darstellungs-Templates. Alternativen wie React oder

Angular wurden verworfen: React hätte mehr manuelle Zustandsarchitektur erfordert, Angular wäre für den Projektumfang überdimensioniert gewesen — und beide hätten weder die Vorlage noch die Vorerfahrung genutzt.

Backend — Spring Boot, Java 21. Die Wahl von Spring Boot war eine bewusste, auf die konkrete Anwendung zugeschnittene Entscheidung: Das Schema ist klar relational, und Spring Boot bietet mit der konventionsbasierten Struktur (Controller → Service → Repository) und **Spring Data JPA** eine deklarative Persistenzschicht, die die vorgegebenen Tabellen direkt auf Entitäten abbildet. **Spring Validation** sichert Eingaben auf Controller-Ebene ab. Auch hier war die Vorerfahrung des Teams ein Vorteil, sodass sich die Arbeit auf die Fachlogik konzentrieren konnte statt auf Boilerplate.

Datenbank — PostgreSQL. PostgreSQL war durch das Schema faktisch vorgezeichnet und passte fachlich sehr gut: native UUID-Primärschlüssel, der JSONB-Typ (mit GIN-Index) für flexible Inhaltsbausteine sowie BYTEA für Binärdaten (Bilder, PDF). Auch mit PostgreSQL hatte das Team bereits Erfahrung.

4 Anforderungen und Rahmenbedingungen

4.1 Funktionale Anforderungen

- F1** Aufgaben über ein Formular mit bewusster Bestätigung erstellen.
- F2** Inhaltsbausteine unterschiedlicher Formate (Text, Bild, PDF) hinzufügen und anzeigen.
- F3** Aufgaben zu geordneten Sequenzen (Lernpfaden) und ungeordneten Sammlungen (Kollektionen) zusammenfassen.
- F4** Horizontale Varianten einer Aufgabe über ein gemeinsames Wurzel-Item darstellen.
- F5** Validatoren (Restriktionen) definieren, speichern und Aufgaben zuweisen.
- F6** Referenzdaten (Autor, Lizenz, Typ, Inhaltstyp) auswählen und neu anlegen.
- F7** Hierarchische Tags zuweisen, anzeigen sowie zur Filterung und Suche nutzen.
- F8** Darstellungs-Templates (XML) je Aufgabe bearbeiten und in einer Vorschau prüfen.

4.2 Nicht-funktionale Anforderungen

- N1 Datenintegrität:** referentielle Konsistenz auf Datenbankebene.
- N2 Typsicherheit:** durchgehend typisierte Schnittstellen zwischen Frontend und Backend.
- N3 Wartbarkeit:** klare Schichtung, austauschbare Datenzugriffsschicht.
- N4 Reaktionsfreudigkeit:** optimistische UI-Aktualisierung für einen flüssigen Arbeitsablauf.
- N5 Portabilität:** reproduzierbarer Start über Container, vorbereitet für den Cluster-Betrieb.

4.3 Rahmenbedingungen

- **Plattform:** Die Anwendung soll auf der E-Learning-Plattform der THM laufen. Diese integriert Dienste über Kubernetes; der Zugriff erfolgt über ein zentrales API-Gateway (Traefik), die Images liegen in der GitHub Container Registry (GHCR).
- **Authentifizierung:** nach Plattform-Vorgabe zunächst zu *mocken*; eine globale Anmeldung (THM-CAS) sollte später folgen.

- **Vorgegebenes Datenmodell:** Das Schema war gesetzt und durfte in seiner Grundstruktur nicht verändert werden.
- **Team und Zusammenarbeit:** Entwicklung im Team mit geschütztem Hauptzweig; jede Änderung wird über Pull-Requests begutachtet.

5 Konzept und Entwurf

5.1 Systemarchitektur

Die Anwendung folgt einer klassischen Drei-Schichten-Architektur mit klarer Trennung von Präsentation, Geschäftslogik und Persistenz. Frontend (Port 8085) und Backend (Port 8080) kommunizieren zustandslos über HTTP/JSON; das Backend greift über JPA auf die PostgreSQL-Datenbank (Port 5432) zu.

Adb.vue (Wurzel)

```
| - AdbToolbar.vue      Aktionen (Aufgabe/Kollektion erstellen)
| - AdbStructure.vue    Strukturbaum + Filterleiste (oben/links)
|   \- AdbTreeItem.vue  rekursive Baum-Knoten (Ordner = Kollektion,
|                       Datei = Aufgabe)
\ - AdbEditor.vue       Detailansicht (rechts)
    | - AdbTagsSection.vue  Tags der Aufgabe
    | - AdbContentList.vue  Inhaltsbausteine (Text/Bild/PDF)
    | - AdbValidatorEditor.vue Validatoren
    \ - AdbVariantsPanel.vue horizontale Varianten
```

```
exerciseStore.ts (Pinia) -> ApiAdapter (Interface)
                           | - AdbApiService      (axios, echte API)
                           \ - DevAdbApiService    (lokale Mock-Daten)
```

Adapter-Muster als Entkopplung. Ein zentrales Entwurfselement ist das Adapter-Muster: Der Store spricht ausschließlich gegen die Schnittstelle **ApiAdapter**. Zwei Implementierungen erfüllen diesen Vertrag — **AdbApiService** für echte HTTP-Aufrufe und **DevAdbApiService** mit lokalen Mock-Daten. Dadurch lässt sich das Frontend vollständig ohne laufendes Backend entwickeln und vorführen (Dummy-Modus). Die Auswahl erfolgt zur Startzeit über eine Umgebungsvariable. Das Backend ist spiegelbildlich in drei Ebenen gegliedert: **Controller** (REST-Endpunkte, Eingabevalidierung), **Service** (Geschäftslogik, Transaktionsgrenzen) und **Repository** (Spring Data JPA). Die Datenübertragung erfolgt über typisierte DTOs; eine gemeinsame Typdatei im Frontend spiegelt die Vertragsstruktur, sodass Abweichungen zwischen Client und Server bereits zur Übersetzungszeit auffallen.

5.2 Datenmodell: die Entitäten

Das Schema umfasst 18 Tabellen in drei logischen Ebenen. Jede wird im Frontend über den Store und die Editor-Komponenten bedient und im Backend durch eine JPA-Entität, ein Repository und (soweit fachlich nötig) einen Service abgebildet.

5.2.1 Referenz-Entitäten (Nachschlagelisten)

Diese Tabellen liefern die auswählbaren Stammdaten. Im Backend sind es einfache Entitäten mit Repository; im Frontend werden sie als Auswahllisten geladen.

- **author** — Autor eines Items oder Inhalts (Anzeige-Deskriptor, optional Mail).

- `license` — Lizenz (z. B. CC-BY), an Item und Inhalt hängbar.
- `item_type` — Typ einer Aufgabe (fachliche Kategorie).
- `item_content_type` — Typ eines Inhaltsbausteins (Text, Bild, PDF...).
- `item_representation_template` — XML-Vorlage, die beschreibt, wie die Inhalte einer Aufgabe angeordnet dargestellt werden.
- `tag` — hierarchisches Schlagwort (Selbstreferenz `parent_tag_id`).
- `validator` — methodische Restriktion (z. B. „muss INNER JOIN enthalten“).
- `modifier` — Regel zur (automatischen) Variantenerzeugung (*modelliert, in dieser Iteration ohne UI — siehe Kapitel 8*).

5.2.2 Kern-Entitäten

item (Aufgabe). Jede Aufgabe ist ein `item` mit Pflichtbezügen zu Autor, Lizenz und Typ. Bemerkenswert: es gibt *kein* eigenes Titelfeld — der Titel lebt als Inhaltsbaustein und ist über die Verknüpfung mit einem `purpose` (z. B. „Aufgabenstellung“) zugeordnet. Die Selbstreferenz `root_item_id` verbindet zusammengehörige Varianten. Im Frontend ist ein Item ein Knoten im Strukturbaum; im Backend kapselt `ItemService` das Anlegen, Ändern, Löschen und die Umwandlung in eine Kollektion.

item_content (Inhaltsbaustein). Trägt entweder strukturierte Daten (JSONB) oder Binärdaten (BYTEA, für Bild/PDF) und besitzt eigene Lizenz, Typ und Autor. Im Frontend wird ein Inhalt im `AdbContentEditor` bearbeitet; Binärdaten werden per Multipart-Upload übertragen.

item_collection (Kollektion). Gruppiert Items. Das boolesche Feld `ordered` unterscheidet geordnete Sequenzen (Sub-Items mit `position` 1, 2, 3...) von ungeordneten Sammlungen (`position` = NULL). Über `parent_item_id` ist eine Kollektion zugleich ein Item und besitzt keine eigene Metadaten-Kopie.

5.2.3 Verknüpfungs-Entitäten (n:m-Beziehungen)

- `item_contents` — ordnet einem Item seine Inhaltsbausteine mit `purpose` zu (eigene Verknüpfungsentität mit zusammengesetztem Schlüssel).
- `item_collection_sub_item` — ordnet einer Kollektion ihre Kind-Items samt `position` zu (die vertikale Beziehung).
- `item_tags` / `item_content_tags` — Tags an Items bzw. an Inhalte.
- `item_validator` — Validatoren an Items.
- `item_modifier` — Modifikatoren an Items (*vorbereitet*).
- `item_content_types` — erlaubte Inhaltstypen je Item.

5.3 Beziehungskonzept: vertikal und horizontal

Das Schema kennt bewusst *zwei orthogonale* Beziehungsarten, die im Projekt klar getrennt behandelt werden:

Vertikal (Sequenz)		Horizontal (Variante)
Bedeutung	aufeinander aufbauende, <i>verschiedene</i> Aufgaben in fester Reihenfolge	<i>dieselbe</i> Aufgabe in mehreren Ausprägungen
Mechanismus	geordnete <code>item_collection</code> + <code>position</code>	gemeinsames <code>root_item_id</code>
Rolle des Items	Behälter (Lernpfad)	einzelne Aufgabe

Diese Trennung ist eine tragende Entwurfsentscheidung: Ein Item spielt zu einem Zeitpunkt genau *eine* Rolle. Beim Umwandeln einer Aufgabe mit Varianten in eine Kollektion warnt die Anwendung daher ausdrücklich, statt beide Beziehungsarten stillschweigend zu vermischen. Konsequenz dazu lässt das Löschen einer Kollektion die enthaltenen Aufgaben unberührt (sie werden nur aus der Kollektion gelöst) und tastet ihren Varianten-Bezug nicht an.

5.4 Referentielle Integrität

Die Fremdschlüssel setzen bewusst unterschiedliche Löschrategien ein:

Beziehung	Strategie	Wirkung
<code>item → author/license/type</code>	RESTRICT	Stammdaten sind gegen Löschen geschützt
<code>item.root_item_id → item</code>	SET NULL	Varianten überleben das Löschen des Wurzel-Items
<code>tag.parent_tag_id → tag</code>	SET NULL	Unter-Tags werden zu Wurzel-Tags
<code>item_collection.parent_item_id → item</code>	CASCADE	die Kollektion verschwindet mit ihrem Item
<code>item_tags, item_contents, ...</code>	CASCADE	Verknüpfungen werden mit Item/Content aufgeräumt

6 Umsetzung

Dieser Abschnitt beschreibt die umgesetzten Funktionen und — wo es das Verständnis fördert — was bei einer Aktion zwischen Frontend, Backend und Datenbank tatsächlich passiert.

6.1 Aufgabe erstellen (durchgängiger Ablauf)

Statt eine Aufgabe sofort beim Klick anzulegen, öffnet die Anwendung einen Formular-Dialog (Typ, Autor, Lizenz, Aufgabentext). Erst mit „Erstellen“ startet der Ablauf: Der **Store** legt die Aufgabe *optimistisch* sofort lokal im Baum an (die UI reagiert unmittelbar) und ruft über den

Adapter nacheinander `POST /items` (Item anlegen), `POST /items/{id}/ contents` (Aufgabentext als Inhalt) und — falls in einer Kollektion — `POST /collections/{id}/items` (Zuordnung) auf. Im **Backend** nehmen `ItemController` und `ItemContentController` die Anfragen entgegen, die zugehörigen Services führen die Fachlogik aus und die Repositories schreiben in PostgreSQL. Die Antwort liefert die echten IDs, die der Store gegen die temporären optimistischen IDs austauscht. Schlägt ein Schritt fehl, gleicht der Store den betroffenen Teilbaum wieder mit dem Datenbankstand ab, sodass keine „Geister-Aufgabe“ zurückbleibt.

6.2 Inhalte in mehreren Formaten

Inhaltsbausteine unterstützen Text (JSON), Bilder und PDF. Binärdaten werden per Multipart-Upload übertragen und im Feld `blob_serialized_content` abgelegt; Bilder erscheinen als Vorschau, PDFs als Download-Link. Jeder Baustein trägt einen `purpose` und eigene Metadaten (Typ, Lizenz).

6.3 Kollektionen und Reihenfolge

Aufgaben lassen sich in Kollektionen gruppieren. Ein Umschalter überführt eine Kollektion zwischen geordnet und ungeordnet; beim Einschalten vergibt das Backend fortlaufende Positionen, beim Ausschalten werden sie auf `NULL` gesetzt (Frontend und Backend werden dabei konsistent gehalten). Die Umsortierung innerhalb einer geordneten Kollektion erfolgt per Drag-and-Drop (`vuedragable`); die Positionen werden serverseitig neu berechnet.

6.4 Horizontale Varianten

Varianten einer Aufgabe teilen sich ein `root_item_id` und werden im `AdbVariantsPanel` angezeigt. Beim Auswählen einer Aufgabe lädt die Anwendung automatisch alle Geschwister-Varianten des gemeinsamen Wurzel-Items. Der oben genannte Warnhinweis verhindert, dass eine Aufgabe versehentlich gleichzeitig Variante und Kollektion wird.

6.5 Validatoren

Validatoren beschreiben methodische Restriktionen. Sie werden gemäß der Projektabgrenzung nur *definiert, gespeichert und Aufgaben zugewiesen*; die Ausführung erfolgt extern. Umgesetzt sind Anlegen, Bearbeiten, Löschen und Zuordnung (`AdbValidatorEditor` → `ValidatorController`).

6.6 Referenzdaten auswählen und anlegen

Autor, Lizenz, Typ und Inhaltstyp werden aus dem Backend geladen und im Editor über Auswahllisten gesetzt. Über eine „+ Neu“-Funktion lassen sie sich direkt neu anlegen; doppelte Namen werden server- und clientseitig abgefangen.

6.7 Hierarchische Tags, Filter und Suche

Tags sind über `parent_tag_id` hierarchisch und werden in Pfadschreibweise dargestellt (z. B. `Mathematik/Analysis/Ableitungen`). Die Zuweisung erfolgt über einen aufklappbaren Baum mit Mehrfachauswahl; feste Regeln gelten beim Anlegen (global eindeutig, keine Leerzeichen, genau ein Elternknoten, kein gleichzeitiges Markieren von Tag und Vorfahr/Nachfahr). Eine gemeinsame obere Filterleiste kombiniert den Tag-Filter (*mit Vererbung*: ein Eltern-Tag findet auch Aufgaben mit Nachfahr-Tags) mit der Text- und Attributsuche (Typ, Autor).

6.8 Darstellungs-Templates

Je Aufgabe kann ein XML-Template die Anordnung der Inhalte beschreiben. Es wird im **CodeMirror**-Editor bearbeitet, gegen die vorhandenen **purposes** validiert (fehlende/überzählige Bausteine werden gemeldet) und in einer Vorschau geprüft.

6.9 Qualitätssicherung

Die Qualitätssicherung ruht auf drei Säulen:

- **Statische Typprüfung:** **vue-tsc** im Frontend und der Java-Compiler im Backend fangen Vertragsverletzungen früh ab.
- **Automatisierte Tests:** Unit-Tests mit **vitest** (Frontend, u. a. für Store-Logik und die neu abgesicherten Fehlerfälle) sowie Mockito-basierte Service- und Controller-Tests im Backend — ohne Bedarf an einer laufenden Datenbank.
- **Strukturvalidierung:** eine reine Funktion prüft den geladenen Baum gegen fachliche Regeln (nur Kollektionen dürfen Kinder haben, jede **root_item_id** muss auflösbar sein).

6.10 Grundlage für den Cluster-Betrieb

Für den Betrieb auf der Hochschulplattform ist die Anwendung als klassischer, synchroner Microservice ausgelegt. Eine CI/CD-Pipeline (GitHub Actions) baut bei jedem erfolgreichen Merge in den Hauptzweig die Container-Images für Backend und Frontend und veröffentlicht sie in der GitHub Container Registry (GHCR). Je Anwendung beschreibt eine **values.yaml**, wie der Dienst im Cluster heißt, wie viele Repliken er benötigt und über welchen Netzwerkpfad er erreichbar ist; **ArgoCD** sollte diese Konfiguration mit dem Cluster synchronisieren (zum tatsächlichen Deployment siehe Kapitel 8).

7 Projektorganisation

7.1 Vorgehen und Versionsverwaltung

Die Entwicklung erfolgte git-basiert mit Feature-Zweigen und Pull-Requests. Der Hauptzweig ist gegen direkte Pushes geschützt; jede Änderung wurde vor dem Zusammenführen von einem anderen Teammitglied begutachtet. Die Arbeit wurde über ein **GitHub-Project-Board** organisiert, auf dem Aufgaben als Tickets durch die Spalten *In Progress*, *Reviewed* und *Closed* wanderten — so waren Stand und Zuständigkeiten jederzeit sichtbar.

*[Screenshot: GitHub-Project-Board (Spalten In Progress / Reviewed / Closed)
hier einfügen]*

7.2 Zusammenarbeit im Team

Über die reine Werkzeugnutzung hinaus prägten regelmäßige Absprachen die Zusammenarbeit:

- **Meetings ein- bis zweimal pro Woche** — je nach Fortschritt und offenen Fragen. Sie dienten der Abstimmung, der Klärung von Schnittstellen und der gemeinsamen Priorisierung.

- **Zwei Pair-Programming-Sitzungen** — für besonders verzahnte oder knifflige Aufgaben, bei denen gemeinsames Arbeiten Fehler früh vermied und Wissen im Team verteilte.

Wiederkehrende Herausforderung war das Zusammenführen länger laufender Feature-Zweige, da mehrere Funktionen dieselben zentralen Dateien (Store, Adapter, Typen) berührten. Als Konsequenz wurde ein Arbeitsablauf mit häufiger, frühzeitiger Integration und konsequenter Typ- und Testprüfung nach jedem Merge etabliert.

7.3 Projektplanung: Phasen und Meilensteine

Das Projekt lief von Ende April bis zur Abgabe am 14. Juli 2026 (Aktivitäts- schwerpunkt im Juni).

Phase	Zeitraum	Inhalt
Konzeption & Einarbeitung	Ende Apr. – Mitte Mai	Team-Aufbau, Analyse des Schemas, Technologieauswahl, Projektstruktur
Neuausrichtung	Mitte Mai	Ausstieg zweier Mitglieder, Umverteilung auf drei Personen
Grundgerüst	Mai	Drei-Schichten-Aufbau, DB-Schema, Docker-Compose, Adapter-Muster, Baum-/Editor-Ansicht
Kernfunktionen	Juni	Inhalte, Kollektionen/Reihenfolge, Varianten, Validatoren, Referenzdaten, Frontend-Backend-Anbindung
Erweiterung & Integration	Juli	hierarchische Tags + Filter, Suche, Repräsentations-Templates, CI → GHCR, Robustheits-Fixes
Abgabe	14. Juli	Dokumentation, Tests, finale Bereinigung

Wesentliche Meilensteine: die abgestimmte fachliche Konzeption, die erste durchgängige Frontend-Backend-Anbindung, die fachliche Vollständigkeit der Kernfunktionen sowie die einsatzbereite Build-Pipeline mit veröffentlichten Images in GHCR.

7.4 Aufgabenverteilung

Die Arbeit war zunächst nach Schwerpunkten aufgeteilt, verlief zum Projektende hin aber zunehmend übergreifend:

Person	Schwerpunkt
[Vorname Nachname] (<i>Projektleitung</i>)	Datenbank und Backend zu großen Teilen, Deployment (CI/CD, GHCR, <code>values.yaml</code>) eigenständig, Koordination; gegen Ende zusätzlich Frontend
[Vorname Nachname]	Schwerpunkt Frontend (Baum, Editor, UI-Komponenten); gegen Ende auch Backend-Aufgaben
[Vorname Nachname]	Schwerpunkt Backend (Endpunkte, Persistenz); gegen Ende auch Frontend-Aufgaben

Anfangs bearbeitete eine Person überwiegend das Frontend und berührte das Backend nicht, während eine andere überwiegend im Backend arbeitete; die Projektleitung verantwortete Datenbank, Backend und das gesamte Deployment eigenständig. Im weiteren Verlauf lösten sich

diese Grenzen auf — teils als Folge der verkleinerten Teamgröße, was zugleich dazu führte, dass alle drei Personen ein gutes Gesamtverständnis des Systems erlangten.

7.5 Abweichungen vom ursprünglichen Plan

Das Projekt startete mit **fünf** Personen. Nach etwa drei Wochen stiegen zwei Mitglieder aus, sodass die verbleibende Arbeit auf **drei** Personen neu verteilt werden musste. Das erhöhte die Last pro Person spürbar und war der wesentliche Grund für die spätere schichtübergreifende Arbeitsweise. Weitere inhaltliche Abweichungen sind in Kapitel 8 zusammengefasst.

8 Grenzen und Ausblick

Die folgende ehrliche Bestandsaufnahme trennt, was umgesetzt wurde, was bewusst zurückgestellt und was aus externen Gründen nicht realisiert werden konnte.

8.1 Nicht realisiert (mit Begründung)

- **Cluster-Deployment.** Die Build-Pipeline (CI → GHCR) und die `values.yaml` je App sind fertig. Das eigentliche Ausrollen im Cluster kam jedoch nicht zustande: Auf die Anfrage, uns in ArgoCD einzutragen, teilte das zuständige Infrastruktur-Team (Gruppe 1) mit, dass es das Projekt *nicht weiterführt*. Ohne diese Registrierung war ein Deployment nicht möglich — ein externer, nicht vom Team zu verantwortender Umstand.
- **Authentifizierung.** Nach Plattform-Vorgabe sollte die Authentifizierung zunächst gemockt und später zentral (THM-CAS) angebunden werden. Die dafür nötigen Vorgaben bzw. das Datenschema wurden von der zuständigen Stelle nicht bereitgestellt, sodass die Umsetzung nicht weiterverfolgt wurde; die Anwendung bleibt ohne Rechteeinschränkung.
- **KI-Funktion.** Eine im Lastenheft angedachte KI-Unterstützung wurde im Projektverlauf — mit Blick auf Umfang und die verkleinerte Teamgröße — nicht implementiert.

8.2 Bewusst zurückgestellt

- **Modifikatoren.** Die Entität und Verknüpfung (`modifier`, `item_modifier`) sind im Datenmodell vorhanden, es gibt aber bewusst *keine* UI. Auf Wunsch des Betreuers wurden die Modifikatoren zunächst ausgeklammert, da noch nicht abschließend geklärt ist, wie die automatische Variantenerzeugung fachlich ausgestaltet sein soll.

8.3 Bekannte technische Grenzen

- **Ein Kollektions-Elternteil je Item.** Die zugrunde liegende Tabelle `item_collection_sub_item` erlaubt theoretisch, dass ein Item in mehreren Kollektionen liegt. Die Baum-Oberfläche stellt ein Item jedoch an genau einer Stelle dar und setzt damit praktisch einen einzigen Kollektions-Elternteil voraus.
- **Umfang des zentralen Stores.** Der Pinia-Store ist mit der Zeit stark gewachsen. Eine Aufteilung in fachliche Composables (Baum/Kollektionen, Inhalte, Tags, Suche) würde die Wartbarkeit weiter verbessern.

8.4 Ausblick

Für kommende Iterationen bieten sich an: das vollständige Cluster-Deployment (sobald eine Infrastruktur bereitsteht), die zentrale Anmeldung (THM-CAS) mit darauf aufbauendem Rech-

temodell, die Ausgestaltung der Modifikatoren, ein weiterer Ausbau der automatisierten Testabdeckung sowie die Modularisierung des Stores.

9 Fazit

Das Projekt setzt das vorgegebene Datenmodell in einer vollständig lauffähigen Drei-Schichten-Anwendung um. Die zentralen fachlichen Konzepte — Metadaten, Inhalte in mehreren Formaten, geordnete und ungeordnete Kollektionen, horizontale Varianten, Validatoren, hierarchische Tags mit Suche sowie Darstellungs-Templates — sind implementiert und über eine benutzerfreundliche Oberfläche zugänglich. Tragende Entwurfsentscheidungen, insbesondere die saubere Trennung von vertikalen und horizontalen Beziehungen und die Entkopplung der Datenzugriffsschicht über ein Adapter-Muster, sorgen für fachliche Klarheit und Erweiterbarkeit. Mit der durchgehenden Typsicherheit, der regelbasierten Strukturvalidierung und der Container-Grundlage besteht eine solide Basis, auf der — sobald die externen Voraussetzungen (Infrastruktur, Auth-Vorgaben) gegeben sind — die verbleibenden Punkte aufsetzen können.

Literatur

- [1] Vue.js — The Progressive JavaScript Framework. <https://vuejs.org>
- [2] Vuetify — Vue Component Framework. <https://vuetifyjs.com>
- [3] Pinia — The Vue Store. <https://pinia.vuejs.org>
- [4] Vue.Draggable. <https://github.com/SortableJS/vue.draggable.next>
- [5] CodeMirror 6. <https://codemirror.net>
- [6] Vite — Next Generation Frontend Tooling. <https://vitejs.dev>
- [7] Spring Boot Reference Documentation. <https://spring.io/projects/spring-boot>
- [8] PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>
- [9] Kubernetes Documentation. <https://kubernetes.io/docs/>
- [10] Argo CD Documentation. <https://argo-cd.readthedocs.io>
- [11] Traefik Proxy Documentation. <https://doc.traefik.io/traefik/>
- [12] THM E-Learning-Plattform — Onboarding-Leitfaden (internes Dokument).

Anhang

A. Lokale Ausführung

```
docker compose up --build
# Frontend: http://localhost:8085   Backend: 8080 (intern)   DB: 5432
# Dummy-Modus (ohne Backend):  npm run dev:dummy
```

B. Deployment-Konfiguration (Auszug values.yaml, Backend)

```
name: pwi-aufgabendatenbank-backend
image:
  repository: ghcr.io/.../pwi-aufgabendatenbank-backend
  tag: latest
replicas: 1
containerPort: 8080
route:
  path: /api
```

C. Screenshots der Anwendung

[Screenshot: Strukturbaum mit Kollektionen und Aufgaben]

[Screenshot: Editor mit Inhalten, Tags und Varianten]

[Screenshot: obere Filterleiste (Tag-Filter + Suche)]

D. Weiteres

- ER-Diagramm des Datenmodells (*hier einfügen*).
- GitHub-Repository: PWI-Aufgabendatenbank.